Experimento 3 · RoPE y coste

Guía orientativa para la memoria · Checkpoints DACSA definitivos · 26/10/2026

Qué comprobamos y qué esperábamos

Usamos los dos modelos entrenados del experimento 2. Primero comprobamos en CPU y FP32 una propiedad de RoPE: con Q y K fijos, desplazar las dos posiciones manteniendo su separación debería conservar el producto escalar, salvo pequeños errores numéricos. Después medimos el forward completo en la RTX 5070 Laptop, en FP16, con lote 1. No había garantía de que RoPE fuese más rápido ni de que consumiera menos memoria.

Qué observamos

La prueba analítica dio un error absoluto máximo de $1{,}21 \times 10^{-6}$ entre los 16 pares Q/K normalizados. Es un error muy pequeño y compatible con lo esperado en FP32: se conserva la propiedad de posición relativa en esta prueba.

Longitud	Learned (ms)	RoPE (ms)	Miles tokens/s (L / R)
64	2,34	2,94	27,4 / 21,8
128	2,38	3,16	53,8 / 42,1
256	2,23	2,82	114,8 / 90,7
512	2,35	2,92	218,6 / 175,6

Son medias de cinco mediciones de diez forwards, tras cinco forwards de calentamiento por longitud y con CUDA sincronizada. Learned fue más rápido en las cuatro longitudes. Con 512 tokens alcanzó unos 218.615 tokens/s frente a 175.629 de RoPE: aproximadamente un 24,5 % más de rendimiento. RoPE a longitud 128 presentó mayor dispersión, por lo que ese punto conviene interpretarlo con cautela.

RoPE tiene menos parámetros: 4.887.040 frente a 5.018.112; elimina 131.072 parámetros posicionales aprendidos. Aun así, a longitud 512 el pico de memoria asignada fue 19,33 MiB para RoPE y 19,08 MiB para Learned. El incremento del forward fue casi igual: 7,500 frente a 7,504 MiB. Menos parámetros no implica necesariamente menor memoria total: también intervienen las cachés, los buffers y los tensores temporales.

Qué podemos concluir y qué no

La propiedad analítica de RoPE se cumple dentro del error numérico, pero no demuestra invariancia del modelo completo al cambiar el contexto. En esta implementación, Learned ofreció mayor velocidad y la memoria del forward fue muy parecida. Medimos memoria asignada por PyTorch, no toda la VRAM de nvidia-smi; el incremento es pico menos memoria previa, sin contar de nuevo el modelo y la entrada. Las entradas eran sintéticas: no evaluamos calidad lingüística, generación token a token ni contextos superiores a 512. Estas cifras describen esta ejecución, no una ventaja universal de una arquitectura.

Fuentes: resultados_exp3/{resumen,parametros,condiciones,validacion_rope}.csv.
Los CSV originales incluyen las desviaciones estándar de las mediciones.